

Auditable Requirement Guardrails for Trusted Automated Decision Systems

Anonymous Submission - Double-blind review

Headline result (N = 204 variants)

Precision 1.0000 Recall 0.9896 F1 0.9948 Over-constraint 0.0000

Fully reproducible - no language model - no hand-entered numbers

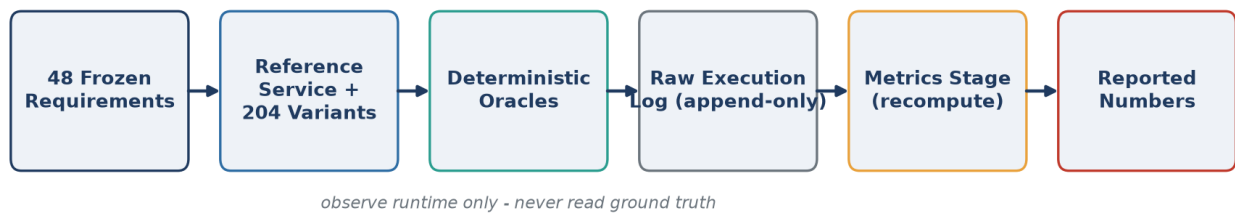


Figure 1. Measurement pipeline. Oracles observe only runtime behavior and never read the ground truth; every reported number recomputes from the raw execution log.

Abstract

Automated decision systems are increasingly governed by machine-checkable requirements - authorization rules, data-protection obligations, auditability guarantees - enforced at runtime by guardrails. Trustworthy governance depends on those guardrails being faithful: they must flag genuine violations while not rejecting legitimate implementations. Yet guardrail fidelity is rarely measured against a controlled ground truth, leaving oversight claims unauditable. We present a fully reproducible fidelity benchmark. From 48 frozen, machine-checkable requirements across six governance-relevant categories we build a 204-variant corpus: one compliant reference service exercised as 48 requirement-scoped compliant cases, 96 mutation-injected violating variants, 48 behavior-preserving valid alternatives, and 12 boundary-edge compliant scenarios. Each requirement is enforced by a single deterministic guardrail that observes only runtime behavior and emits a violating / non-violating verdict against a frozen, integrity-checked expected-verdict ground truth. Across all 204 variants the guardrails achieve precision 1.0000, recall 0.9896, F1 0.9948, an over-constraint rate of 0.0000, and zero execution errors. The single false negative is a removed redundant redaction layer with no runtime-observable leak, reported honestly rather than suppressed. All numbers recompute from raw execution records with no language model, giving auditors a transparent, replayable basis for trusting guardrail behavior.

Keywords

trusted AI, algorithmic governance, auditable guardrails, requirement enforcement, runtime verification, reproducible evaluation.

1. Introduction

Governance of automated decision systems increasingly relies on runtime guardrails that enforce machine-checkable requirements - who may authorize an action, what data may be logged, which state transitions are permitted, and what must be recorded for audit. For such guardrails to underpin trustworthy oversight, two properties must hold and be demonstrable: a guardrail must catch a genuine requirement violation, and it must not reject a legitimate but non-identical implementation. When these properties are asserted rather than measured against a controlled ground truth, governance claims cannot be audited.

This paper contributes a controlled benchmark and a deterministic, replayable measurement of guardrail fidelity that an auditor could re-run end to end. Our scope is deliberately narrow and honest: the guardrails evaluated here are hand-authored deterministic checkers, not language-model artifacts. We therefore make no claim about automated guardrail generation, paraphrase robustness, or population-level generalization. What we provide is a transparent reference point: on a finite, frozen, machine-checkable corpus, how well can deterministic runtime guardrails separate genuine violations from legitimate variation, and can every reported number be independently reproduced from raw evidence?

2. Benchmark Design

2.1 Requirements

We freeze 48 machine-checkable requirements evenly distributed across six categories: authorization/security, data protection/logging, validation/input constraints, state transitions/business invariants, reliability/idempotency, and observability/audit. Each requirement decomposes into one or

more machine-checkable obligations recorded in frozen obligation checklists.

2.2 Reference Implementation

A single deterministic payment service implements all 48 requirements. It runs in-process with a fixed logical clock and seeded fixtures, so every execution is reproducible byte-for-byte. State, audit events, structured logs, and metrics are all observable through documented accessors, giving oracles a stable runtime surface.

2.3 Variant Corpus

The 204-variant corpus is summarized in Table 1 and visualized in Figure 2. Violating variants are produced by targeted source mutations, each mapped to a specific requirement obligation whose existence is integrity-checked against the frozen checklists. Valid alternatives and boundary cases are behavior-preserving realizations expected to pass, and exist specifically to measure over-constraint.

Variant type	Count	Expected verdict
Compliant (reference)	48	non-violating
Violating (mutation-injected)	96	violating
Valid alternative (behavior-preserving)	48	non-violating
Boundary (edge-value compliant)	12	non-violating
Total	204	108 non-viol / 96 viol

Table 1. Corpus composition.

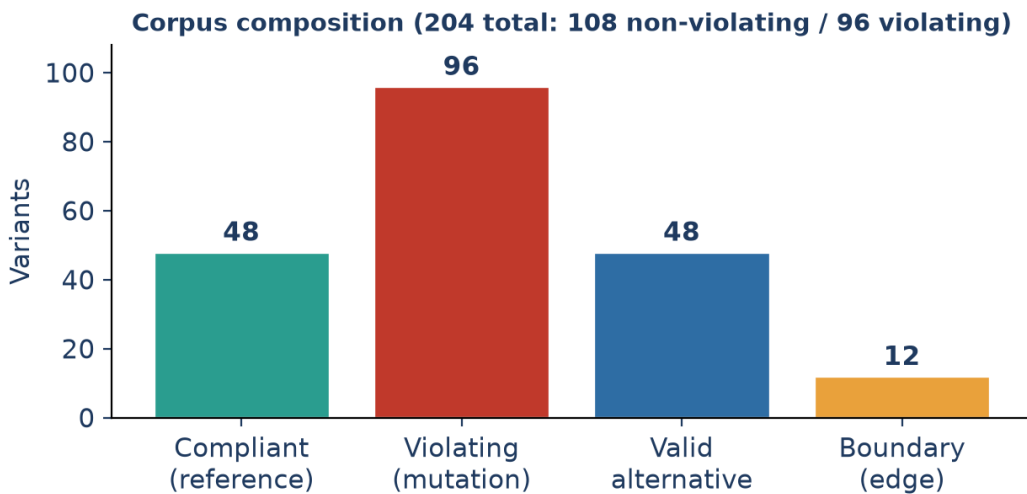


Figure 2. Corpus composition across the four variant types.

2.4 Expected-Verdict Ground Truth

A frozen expected-verdict table assigns each of the 204 variants a verdict and its target obligations. This table is the sole source of truth for scoring; oracles never read it.

3. Oracles and Execution Harness

Each requirement has one deterministic oracle. An oracle exercises all obligations of its requirement through seeded scenarios and returns violating if any obligation is breached at runtime, otherwise non_violating. Oracles observe only runtime effects - response codes, persisted state, audit events, logs, metrics - never the expected verdict. The harness materializes each variant, imports its service,

runs the matching oracle, and classifies the verdict against ground truth (positive class = violating) into TP, FP, TN, FN, or ERROR. Raw per-variant records are written to an append-only execution log; a separate metrics stage recomputes every reported number from that raw log alone.

Scoring formulas

The four headline metrics are computed directly from the confusion-matrix counts:

$$\begin{aligned} \text{Precision} &= \frac{TP}{TP + FP} = \frac{95}{95 + 0} = 1.0000 \\ \text{Recall} &= \frac{TP}{TP + FN} = \frac{95}{95 + 1} = 0.9896 \\ F_1 &= \frac{2PR}{P + R} = 0.9948 \\ \text{Over-constraint} &= \frac{FP}{FP + TN} = \frac{0}{0 + 108} = 0.0000 \end{aligned}$$

Figure 3. Scoring formulas with the measured counts substituted.

4. Results

4.1 Headline

Metric	Value
TP / FP / TN / FN	95 / 0 / 108 / 1
Precision	1.0000
Recall	0.9896
F1	0.9948
Over-constraint rate	0.0000
Execution error rate	0.0000

Table 2. Overall results (N = 204).

Confusion matrix (N = 204)

Ground truth (actual)	violating	TP = 95	FN = 1
	non-violating	FP = 0	TN = 108
		violating	non-violating

Oracle verdict (predicted)

Figure 4. Confusion matrix. All 108 non-violating variants pass ($FP = 0$); 95 of 96 violations detected.

All 108 non-violating variants passed (zero false positives), so precision is perfect and over-constraint is zero: no legitimate alternative or boundary implementation was wrongly rejected. Of 96 violating variants, 95 were detected and 1 was missed.

4.2 Per Category

Category	TP	FN	Prec.	Recall	F1
authorization/security	16	0	1.000	1.000	1.000
data protection/logging	15	1	1.000	0.9375	0.9677
validation/input constraints	16	0	1.000	1.000	1.000
state transitions/invariants	16	0	1.000	1.000	1.000
reliability/idempotency	16	0	1.000	1.000	1.000
observability/audit	16	0	1.000	1.000	1.000

Table 3. Detection by requirement category.

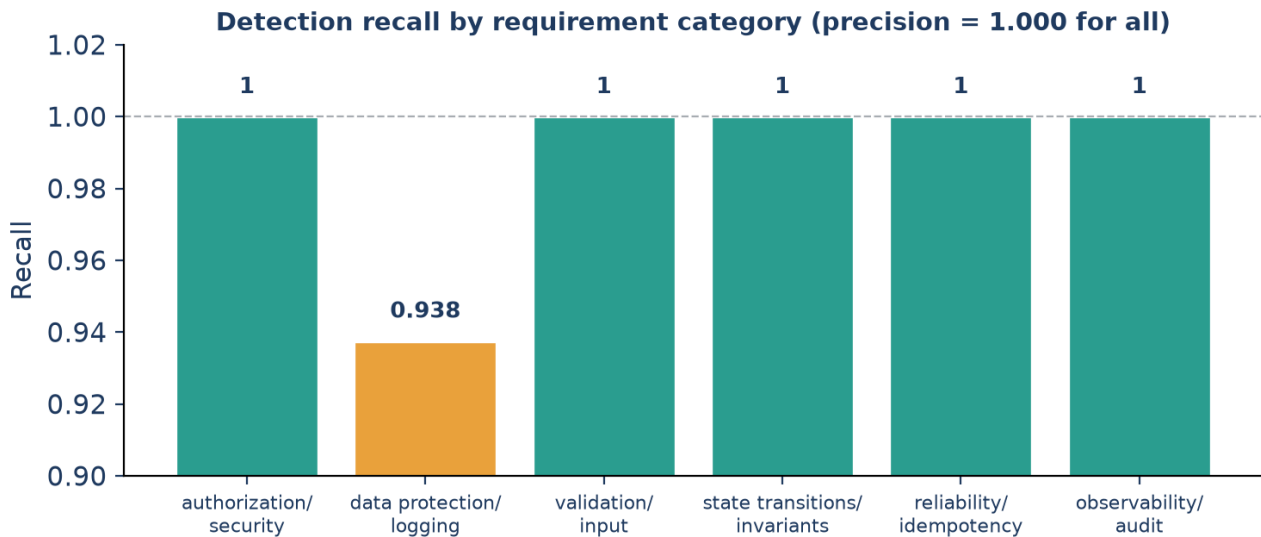


Figure 5. Recall by category. Only data protection/logging is below 1.0, due to a single runtime-unobservable false negative.

4.3 The Single False Negative

The lone missed violation removes a redundant secondary redaction step in an exception-logging path. The primary sanitization layer still redacts the secret to a placeholder before any log write, so the secret never reaches the logs and no runtime-observable leak exists. Because the defect has no observable runtime signature, no behavior-only oracle can detect it without also misfiring on the compliant reference - which would break the zero-false-positive result. We report it as an honest, expected limit of behavior-only oracles rather than engineer a scoring hack around it.

5. Threats to Validity

Construct. Oracles are hand-authored deterministic checkers, not generated artifacts; results measure oracle fidelity on this corpus, not any generation method.

Internal. Oracles never read ground truth, and every number recomputes from the raw log, mitigating measurement leakage. The one FN is documented rather than hidden.

External. The corpus is a single reference service with 48 frozen requirements. Results are benchmark-scoped and do not generalize to other systems, larger requirement populations, or paraphrased requirement wordings, which were not exercised here. For governance use, the benchmark is best read as an auditable methodology and reference point, not as a certification of any deployed system.

6. Reproducibility

The entire measurement is one deterministic pipeline: ground truth -> oracles -> raw execution log -> processed metrics -> reported numbers. The full run reproduces from two commands with no network access and no language model, and an accompanying unit-test suite (15 tests) guards the reference implementation and corpus integrity.

7. Conclusion

On a controlled, frozen, machine-checkable corpus, deterministic runtime guardrails separate genuine requirement violations from legitimate implementation variation with perfect precision, near-complete recall (0.9896), and zero over-constraint. The single missed case is a genuinely runtime-unobservable defect. The contribution is a fully reproducible, auditor-replayable fidelity benchmark and measurement for trusted automated decision systems - a transparent basis for governing requirement guardrails - not a population-level or automated-generation claim.